

kmhelpers: A Python toolkit for automated management of genomic indexes

Sébastien Bellenous¹ and Pierre Peterlongo ¹

¹ Genscale, Univ. Rennes, Inria, CNRS, IRISA - UMR 6074, Rennes, F-35000 France

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#)).

Summary

Large-scale genomic sequence search has become a central challenge in modern bioinformatics [REFSTODO]. A family of scalable methods uses Bloom filters (BF) (Bloom, 1970). At query time, this enables assigning the queried k -mers to the set of datasets they belong to. Building such indexes on a vast set of genomic samples requires several steps. First, determine the length of each sample (in terms of its number of distinct k -mers). Second, given a user-defined false positive rate, distribute input samples into distinct groups, each BF of a group having the same fixed size. The sizes of BF of groups have to be optimized to reduce the final whole index size. Third, given computational constraints, the whole construction is split into jobs, each generating sub-parts of the index, and a final step merges these sub-parts.

Those steps require expertise, and any error at any stage can invalidate downstream results and waste significant computation time.

//: # such as those produced by kmindex (Téo Lemané & others, 2024), : enumerating sample files, counting k -mers, selecting appropriate Bloom-filter parameters, generating index definitions, orchestrating build jobs, and compressing outputs for long-term storage. Each step requires expertise, and errors at any stage can invalidate downstream results and waste significant computation time.

We propose kmhelpers, an open-source Python toolkit that automates this entire workflow for building (and querying) indexes created using kmindex (Téo Lemané & others, 2024). kmhelpers provides both a command-line interface (CLI) and a Python API, covering every stage of the k -mer index lifecycle.

Statement of Need

k -mer based sequence databases built with tools like kmindex and kmtricks (Thibault Lemané et al., 2022) are used to address large-scale questions in genomics, metagenomics, and population studies. However, the gap between having the raw indexing software and running a reproducible, end-to-end indexing workflow is significant. Researchers face recurring pain points:

Build steps - sample discovery and k -mer counting (`list`), - Bloom-filter span profiling and grouping (`profile`), - index definition generation (`compose`), - upfront validation of sample files, available disk space and memory with generation of ready-to-execute pipeline scripts (`plan`), - index building from definition files (`apply`), - sequence querying (`query`), - ZSTD-based index (block) compression (`compress`) (`?`), - registry management (`registry`).

Multi-step workflows can be described as declarative YAML pipelines (`pipeline`) and executed in a single command.

Parameter selection. Bloom filter-based indexes require choosing a filter size (or *span*) per sample. Choosing this parameter incorrectly leads to either storage consumption or unacceptably high false-positive rates. *kmhelpers* automates this with its `profile` command, which reads *k*-mer count distributions produced by *ntCard* (Mohamadi et al., 2017) and assigns each sample to the smallest span that keeps the false-positive rate below a user-defined threshold.

Workflow orchestration. Building a large index may involve hundreds to millions of samples spread across multiple sub-indexes. Managing these build jobs manually is error-prone. *kmhelpers* introduces a declarative index definition format (YAML) and two complementary commands: `plan`, which validates sample files, available disk space and memory upfront and emits ready-to-execute pipeline scripts; and `apply`, which reads the definition files and runs the build, with support for span-level and name-level filtering.

Together, these features make *k*-mer indexing workflows accessible to researchers who are not experts in the underlying data structures, while remaining flexible enough for large-scale production use.

State of the Field

Several tools address large-scale sequence search using *k*-mer based data structures. BIGSI (Bradley et al., 2019) and COBS (Bingmann et al., 2019) use compressed Bloom filter matrices to answer presence/absence queries across collections of sequencing experiments. HowDeSBT (Harris & Medvedev, 2020) and Mantis (Pandey et al., 2018) use sequence Bloom trees and counting quotient filters, respectively, to support similar queries. *kmindex*, which *kmhelpers* wraps, builds on the *kmtricks* counting pipeline and is designed for efficient querying of large sequence collections.

kmhelpers does not compete with any of these approaches at the algorithmic level. Its contribution is orthogonal: it provides the workflow automation, parameter optimisation, and index lifecycle management that are necessary to deploy *kmindex*-based databases in practice, but are absent from the core tool. Similar automation layers exist for other complex bioinformatics pipelines (e.g., Snakemake (Köster & Rahmann, 2012) workflows wrapping alignment or assembly tools), but no dedicated automation layer existed for *kmindex* prior to *kmhelpers*.

Design and Implementation

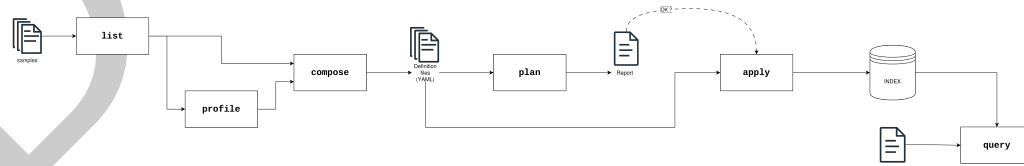


Figure 1: Overview of the *kmhelpers* workflow. `list` enumerates sample files and counts *k*-mers; `profile` analyses the count distributions to assign each sample to a Bloom-filter span and recommend index groupings. Both outputs feed into `compose`, which generates YAML index definition files. `plan` then validates sample paths, available disk space and memory, and emits a ready-to-execute pipeline script; the resulting report is reviewed before committing to the build. `apply` reads the definition files and invokes *kmindex* to construct the index. Finally, `query` searches the built index against user-provided sequences and returns ranked results.

[[TODO]]

kmhelpers is implemented in Python (3.8) and distributed via Conda with automatic installation of its bioinformatics dependencies (*kmindex*, *ntCard*). The CLI is built with Click

(Ronacher & Click Contributors, 2023) and the package exposes a public Python API covering all CLI functionality.

Acknowledgements

The authors thank Téo Lemane for developing kmindex and for his responsiveness in addressing feature requests and issues raised during the development of kmhelpers. We acknowledge the GenOuest core facility (<https://www.genouest.org>) for providing the computing infrastructure. The work was funded by the Inria Challenge “OmicFinder” (<https://project.inria.fr/omicfinder/>).

AI Usage Disclosure

AI assistance was used for constrained tasks (drafting, editing, code suggestions) under strict human review at every stage. AI provider used: Claude (Anthropic, 2025).

References

- Bingmann, T., Bradley, P., Gauger, F., & Iqbal, Z. (2019). COBS: A compact bit-sliced signature index. *String Processing and Information Retrieval (SPIRE 2019)*. https://doi.org/10.1007/978-3-030-32686-9_21
- Bloom, B. H. (1970). Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7), 422–426.
- Bradley, P., Den Bakker, H. C., Rocha, E. P. C., McVean, G., & Iqbal, Z. (2019). Ultrafast search of all deposited bacterial and viral genomic data. *Nature Biotechnology*, 37(2), 152–159. <https://doi.org/10.1038/s41587-018-0010-1>
- Harris, R. S., & Medvedev, P. (2020). Improved representation of sequence Bloom trees. *Bioinformatics*, 36(3), 721–727. <https://doi.org/10.1093/bioinformatics/btz662>
- Köster, J., & Rahmann, S. (2012). Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19), 2520–2522. <https://doi.org/10.1093/bioinformatics/bts480>
- Lemane, Thibault, Medvedev, P., Chikhi, R., & Peterlongo, P. (2022). Kmtricks: Efficient and flexible construction of Bloom filters for large sequencing data collections. *GigaScience*, 11, giac029. <https://doi.org/10.1093/gigascience/giac029>
- Lemane, Téo, & others. (2024). Indexing and real-time user-friendly queries in terabyte-sized complex genomic datasets with kmindex and ORA. *Nature Computational Science*, 4(2), 104–109. <https://doi.org/10.1101/2023.05.31.543043>
- Mohamadi, H., Khan, H., & Birol, I. (2017). ntCard: A streaming algorithm for cardinality estimation in genomics data. *Bioinformatics*, 33(9), 1324–1325. <https://doi.org/10.1093/bioinformatics/btw832>
- Pandey, P., Almodaresi, F., Bender, M. A., Ferdman, M., Johnson, R., & Patro, R. (2018). Mantis: A fast, small, and exact large-scale sequence-search index. *Cell Systems*, 7(2), 201–207.e4. <https://doi.org/10.1016/j.cels.2018.05.021>
- Ronacher, A., & Click Contributors. (2023). *Click: Python composable command line interface toolkit*. <https://click.palletsprojects.com>